

Azure Databricks & PySpark for Data Engineering

Agenda

- Introduction to Big Data & Spark
- Issue with handling/processing of big data:
 - Apache Spark
 - Hadoop vs Spark detailed information
- Hadoop vs Spark Misconceptions Clarification
 - Apache Spark Ecosystem and Architecture
 - Azure Databricks Overview
 - PySpark Fundamentals
 - Spark Optimization & Performance Tuning
- Azure Databricks Workspace Hands-On work

Introduction to Big Data & Spark

- **Big Data:**

1. **Velocity:** The interval at which data is arriving (1 sec, 1 hour)
2. **Variety:** Structured, Semi-structured, Unstructured
3. **Volume:** 5 GB, 10 GB, 50 GB, 10 TB

It means that if we receive some variety of data at a certain velocity of certain volume then that data is called as Big Data.

- **ETL:** Was used before Data Lake, where we were getting limited amount of data which can be processed and stored simultaneously.
- **ELT:** It was introduced after Data Lakes as now we are getting huge amounts of data continuously so, it's a bit difficult to process it parallelly and then store. So, what we do is to store the data as it arrives and then transform it as per our requirements or availability.

Issue with handling/processing of big data:

1. **Storage:** Where will we store this much data
2. **Processing (RAM / CPU):** How will we process this huge amount of data.

To resolve the above-mentioned issues, 2 approaches were introduced:

1. Monolithic approach: A single system to process everything which can be up scaled to a desired limit (Vertical scaling).

- **Problems:**
- Heating issue (Resource Overload): it can be up scaled to a certain limit.
- Expensive – It will increase the cost to maintain and will be a waste of resources as RAM/Storage/CPU needs to be replaced with newer ones.
- Low availability – If it fails, then everything goes down.

2. Distributed approach: Multiple machines to process the data (Horizontal scaling).

- Benefits over the other one:
- Economical – Cheaper parts to upscale as individual machines need to be added.
- High availability – not a single point of failure.

Apache Spark

- Apache Spark is a unified computing engine and set of libraries for parallel data processing on computer cluster.
- Spark is a **computing engine** and not a storage. It processes everything in RAM.
- **Unified:** Spark is designed to support wide range of task over the same computing engine. For example: Data Scientist, Data Analyst, and Data Engineers, all can use the same platform for their analysis, transformation, or modelling requirements.
- **Computing engine:** Used to process data and it can connect with different data sources like S3, ADLS etc.
- **Parallel data processing:** Distributing a set of tasks across multiple worker nodes.
- **Computer Cluster:** Follows master-slave architecture.

HADOOP vs SPARK COMPARISON

	Hadoop	Spark
What is it?	Open-source framework for distributed data storage and processing	Open-source framework for in-memory distributed data processing and app development
Initial release	2006	2014
Supported languages	Java	Scala, Java, Python, R
Processing methods	Batch processing, using hard discs to read/write data	Batch and micro-batch processing in RAM
Built-in capabilities	✓ File system (HDFS) ✓ Resource management (Yarn) ✓ Processing engine (MapReduce)	✓ Processing engine (Spark Core) ✓ Near real-time processing (Spark Streaming) ✓ Structured data processing (Spark SQL) ✓ Graph data management (GraphX) ✓ ML library (MLlib)
Best fit for	Delay-tolerant processing tasks, involving huge datasets	Almost instant processing of live data and quick analytics app development
Real-life use cases	✓ Enterprise archived data processing ✓ Sentiment analysis ✓ Predictive maintenance ✓ Log files analysis	✓ Fraud detection ✓ Telematics analytics ✓ User behavior analysis ✓ Near real-time recommender systems ✓ Stock market trends prediction ✓ Risk management

Hadoop vs Spark

	Hadoop	Spark
Performance	Hadoop is slower than spark because it writes the data back to disk and read again from disk to memory. Process: It reads the data from disk and performs map-reduce operations and then write the result to disk. Then for the next iteration it reads the data again from disk and then performs map-reduce and write again to disk which makes it slower.	Spark is faster than Hadoop because it performs all the computation in memory. Process: It reads the data from disk and the executor performs some transformations on it and then stores the result in cache memory (RAM) and uses it for further processing.
Batch / Streaming	Built for batch data processing.	Built for batch as well as streaming data processing.
Ease of Use	Difficult to write code in Hadoop. Hive was built to make it easier (SQL Type Query).	Easy to write and debug code. We have interactive shell to develop and test. Spark provides high (DataFrame / SQL) and low-level APIs (RDD)
Security	Uses Kerberos authentication and ACL authorization. YARN: It is a resource manager which controls the compute resources (Cluster). It uses Kerberos for security and broadcasts the key to each node	Doesn't have solid security feature. HDFS: ACL YARN: Same will be used in Spark as well to manage compute resources.

	on cluster generated by Kerberos so that it can access the HDFS files. Kerberos: Generates a token for a defined period (default: 10 hours) to authenticate while executing any HDFS commands on a Hadoop Cluster.	For security, it relies on the above-mentioned techniques from Hadoop.
Fault Tolerance	It has block of data and replication factor to handle the failure. Block size: 128 MB each. It means that a given data will be divided into blocks of 128 MBs each and based on the replication factor the data will be replicated across the other nodes for high availability. Example: If replication factor is 4 then each block will be replicated to 4 nodes. Now if a particular node fails then the same data can be referred from other 3 available nodes.	Uses DAG to provide fault tolerance. A DAG is created for each given task. Now consider that DAG has processes from 1 till 10 and it fails in process 5 then the cluster stores the last successful state somewhere for reference and will restart from that point itself when invoked again. Refer to the below mentioned link for explanation.

Hadoop vs Spark

- Misconceptions:

1. Hadoop is a database :– NO

Hadoop is a **Big Data framework/ecosystem** used for: Storage (HDFS) , Distributed processing, Resource management. It's like a platform/framework for handling large-scale data.

2. Spark is 100 times faster than Hadoop:–

Spark can be **up to 100x faster** in some scenarios, especially: In-memory processing , Iterative algorithms, Repeated computations ; But NOT always.

3. Spark processes data in RAM but Hadoop doesn't:- It's partially wrong

Spark prefers processing in memory for speed.

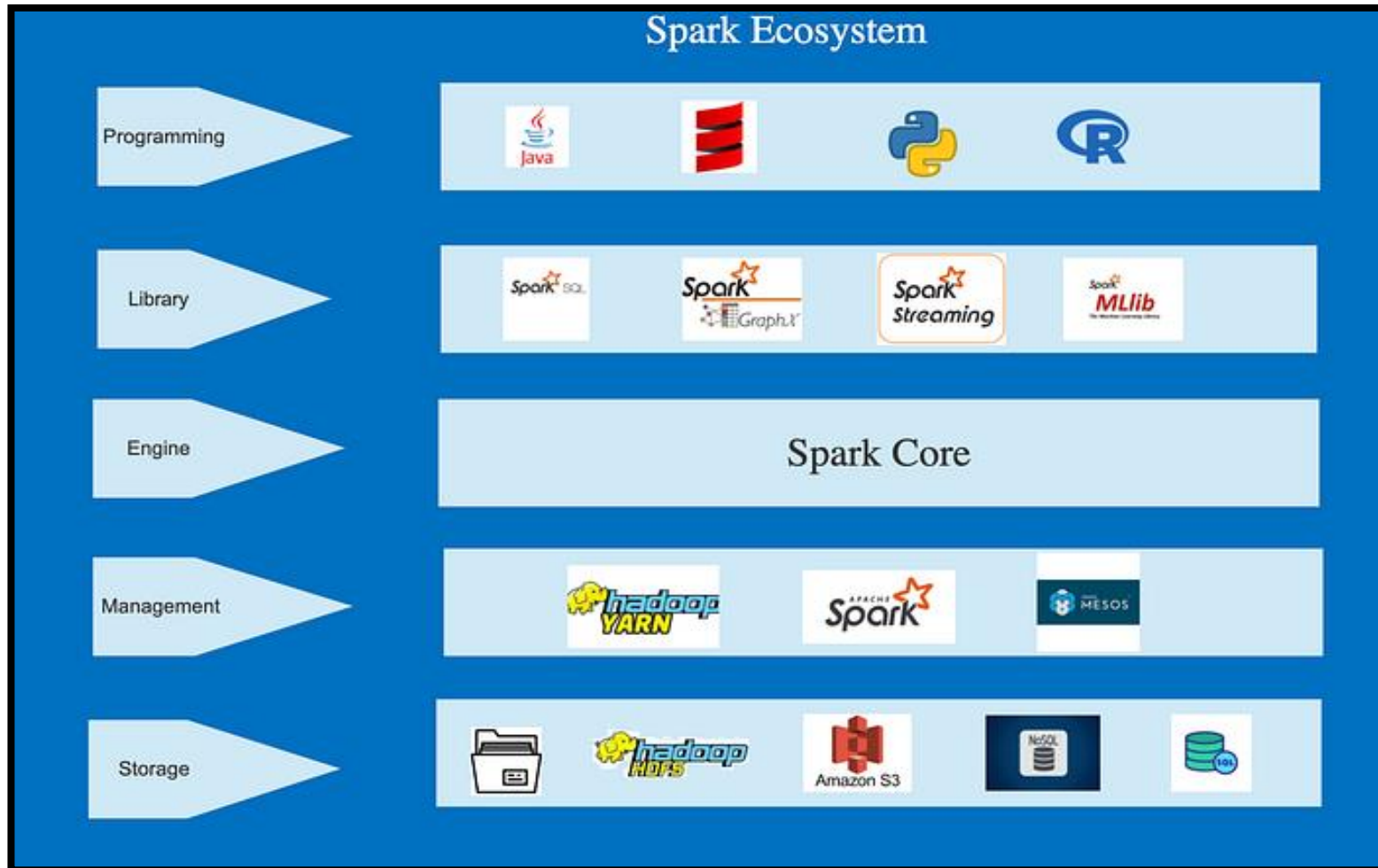
But Spark can also use disk when data is too large.

Hadoop MapReduce mainly writes intermediate data to disk after each stage, which makes it slower.

2. Apache Spark Architecture:

1. Spark Ecosystem Overview

Spark Ecosystem : Collection of libraries/modules built on top of Spark Core for different types of data processing. Different business needs require different processing and Spark provides separate modules.



Engine : Spark Core: The Basic core component of Spark ecosystem on top of which the entire ecosystem is built. DAG Scheduler, Task Scheduler, Memory Management, Fault Tolerance, Execution Engine, Executor coordination

Management : Spark cluster can be managed by Hadoop YARN, Mesos or Spark cluster manager.

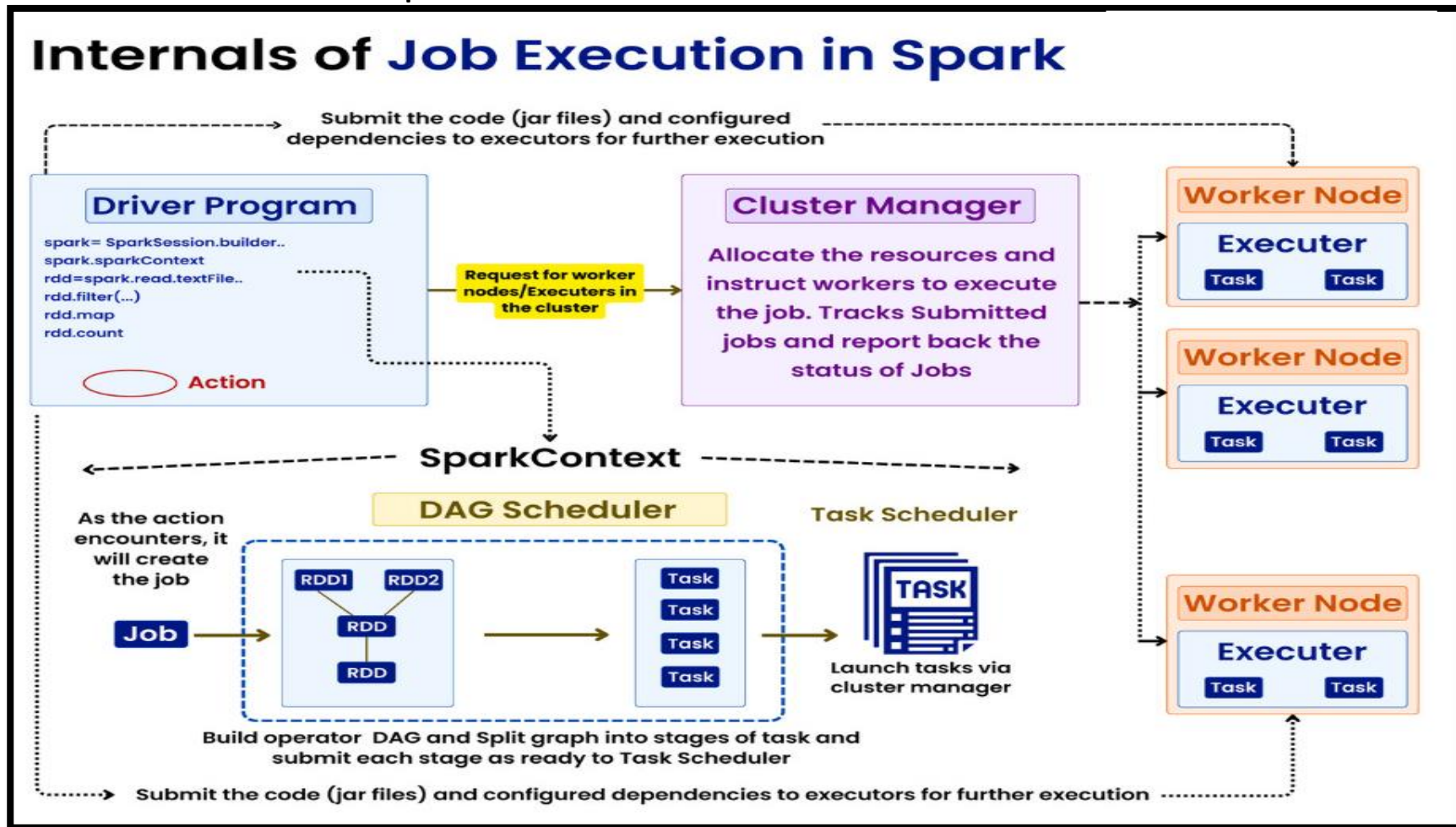
Library: Spark ecosystem comprises of Spark SQL (for running SQL like queries on RDD or data from external sources), Spark Mlib (for ML), Spark Graph X (for constructing graphs for better visualisation of data), Spark streaming (for batch processing and streaming of data in the same application)

Programming: Python, Java, Scala and R

Storage : Data can be stored in HDFS, S3, local storage and it supports both SQL and NoSQL databases.

2. Apache Spark Architecture:

2. Spark Architecture Overview



- Spark follows a **Master (Driver) – Slave/Worker (Executor)** architecture.
- Cluster Managers like **YARN, Kubernetes, Mesos** manage cluster resources.
- Resource Manager allocates CPU, memory, executors, and driver resources to Spark applications.

Working Flow

- Spark Applications consist of a driver process and a set of executor processes.
- The driver process runs your `main()` function, sits on a node in the cluster, and is responsible for three things:
 1. maintaining information about the Spark Application;
 2. responding to a user's program or input; and
 3. analyzing, distributing, and scheduling work across the executors (defined momentarily).

The **driver** process absolutely essential — it's the heart of a Spark Application and maintains all relevant information during the lifetime of the application.

The **executors** are responsible for actually executing the work that the driver assigns them.

This means, each executor is responsible for only two things:

1. executing code assigned to it by the driver
2. and reporting the state of the computation, on that executor, back to the driver node.

1. Application Submission

- Developer submits Spark application using spark-submit.
- Application requests resources like Driver Memory, Executor Memory,
- Number of Executors, CPU Cores.

2. Driver Creation

- Resource Manager allocates a worker node for Driver creation.
- Application Master container gets created.

For Python applications:

PySpark Driver + JVM Driver are created.

- Spark internally runs on JVM because Spark is written in Scala.

3. Executor Creation

- Driver requests executors from Resource Manager.
- Worker nodes are assigned as Executors.
- Executors process distributed tasks in parallel.

PySpark Fundamentals: RDD, DataFrame & Dataset

RDD (Resilient Distributed Dataset)

- Lowest-level distributed data structure in Spark.
- Stores data across multiple nodes in a cluster.
- Supports fault tolerance and parallel processing.
- Data is distributed as partitions across executors.
- Requires more manual coding and optimization.

Advantages

- Fine-grained control over data processing
- Suitable for unstructured data and low-level transformations

Limitations

- No schema support
- No Catalyst Optimizer
- Slower compared to DataFrames/Datasets

DataFrame

- Distributed collection of data organized into rows and columns.
- Similar to a table in SQL or Pandas DataFrame.
- Provides schema-based structured processing.
- Optimized using Spark Catalyst Optimizer and Tungsten Engine.

Advantages

- Faster processing
- Less memory usage
- Easy SQL integration
- Supports large-scale ETL pipelines

Widely Used Because

- Better performance
- Simpler syntax
- Easy integration with SQL and BI tools
- Preferred for Data Engineering workloads

Dataset

- Combination of RDD + DataFrame features.
- Provides compile-time type safety and object-oriented programming support.
- Mainly used in Scala and Java.

Advantages

- Type safety
- Better error detection during compilation
- Optimized execution

Limitations

- Not fully supported in Python (PySpark)
- More complex than DataFrames

- **DataFrames are most widely used in industry:- Reasons:**

1. High performance due to Catalyst Optimizer
2. Easy coding and maintenance
3. Better readability
4. Optimized memory management
5. Best suited for ETL, Analytics, and Big Data processing

Transformations and Actions

Transformations: When we do some processing on data like SELECT, FILTER, GROUP BY, or JOIN.

- Types of transformations:
- 1. **Narrow:** Which do not require data movement between partitions from other executors.
- Example: Filter, Select, Union, Map.
- 2. **Wide:** Which require data movement between partitions from other executors (expensive operation).
- Example: Group By, Join, Distinct, Repartition.

- **Actions:** When we collect the result using commands like COLLECT, COUNT, or DISPLAY, reading a file using spark, INFERSHEMA while reading CSV.
- Spark follows **Lazy Evaluation methodology**. It will not evaluate the transformations until unless we execute any action command. Once actions are triggered, a new job is created, and that job returns the data back to the driver.
- NOTE: The data to be collected back to the driver should not exceed the driver's memory limit otherwise the job will fail (Error: Driver Out of Memory Exception) as the driver will not have sufficient memory to hold the data.
- Example: Driver is of 10 GB RAM, but we are trying to return 15 GB of data from a job back to driver.
- Potential interview questions:
 1. What is transformation and how many types of transformation do we have? – Narrow, Wide
 2. What happens when we use GROUP BY or JOIN in transformation? – Data Movement across different nodes/executors (Wide transformation)
 3. How are jobs created in PySpark? – By trigger of actions
 4. What is Lazy Evaluation

Spark Execution Flow

1. Logical Plan

User writes: Read Data, Filter, GroupBy, Aggregate

2. Optimized Logical Plan (Apply filter before expensive aggregation)

Optimizes query: Pushes filters early, Removes unnecessary columns, Reorders operations

3. Physical Plan: How exactly to execute

Hash Aggregation, Sort Aggregation, Shuffle Exchange

4. DAG Scheduler: Spark converts physical plan into: DAG stages

Stage Type	Meaning
Narrow Transformation	No shuffle
Wide Transformation	Shuffle required

PySpark Code → Logical Plan → Optimized Logical Plan → Physical Plan → DAG Creation → Task Execution

DAG: Directed Acyclic Graph

- Spark internally creates a graph of transformations before execution.
- Spark is Lazy evaluated meaning:- Transformations are NOT executed immediately Spark first builds an execution plan.
- **DAG Helps Spark:** Spark optimizer can:
 - Rearrange operations
 - Minimize shuffling
 - Optimize execution
 - Parallelize tasks

Azure Databricks Overview

- Create Notebook, Attach Cluster, Upload Dataset, Execute PySpark Code
- File Handling using DBFS
- ADLS Gen2 Integration Basics

Spark Optimization & Performance Tuning

- Partitioning, Shuffle, Caching, Broadcast Join, Predicate Pushdown
- repartition() vs coalesce(), Narrow vs Wide Transformations
- Spark Lazy Evaluation, DAG Optimization, Catalyst Optimizer Overview
- Data Skew Handling Basics, Avoiding Excessive Shuffles
- File Format Optimization: Parquet, Delta over CSV
- Efficient Joins: Broadcast Join for Small Tables, Join Optimization
- Caching & Persisting Frequently Used DataFrames
- Partition Pruning, Bucketing Overview
- Small File Problem Overview, OPTIMIZE & ZORDER Concept
- Cluster Optimization Basics: Autoscaling, Executor Memory, Cluster Sizing
- Adaptive Query Execution (AQE) Overview
- Monitoring using Spark UI: Stages, Jobs, Executors, Shuffle Read/Write
- Best Practices: Avoid collect(), Avoid unnecessary repartition(), Use select() instead of select *
- Performance Tuning Goals: Faster Execution, Reduced Memory Usage, Cost Optimization, Efficient Resource Utilization

Thank you!